

易可思科技 AI工程師

Coderbyte 測驗 — 考前速讀重點卡

精簡版：條列重點＋題型對策，考前 30 分鐘可速覽完畢

完整原理／生活比喻／流程圖解／逐步範例，請見另一份 ecostek_full_guide.pdf（完整教學講義）

章節	主題	這次測驗可能性
一	HTTP／Web 通訊	高——考古題已出 redirect
二	作業系統與併發	高——考古題已出 Process/Thread
三	Git 版本控制	高——考古題已出 branch/HEAD
四	資料庫 SQL/NoSQL/Redis	中高
五	框架與非同步	中高
六	系統設計	中——開放題常客
七	AI/LLM/Agent/Prompt	高——職缺核心
八	程式邏輯/演算法	高——Coding Challenge
九	後端安全	中
十	機器學習/深度學習	中高——面試延伸
十一	Docker	中
十二	CI/CD/DevOps	中低

一 HTTP／Web 通訊基礎

無狀態協定、Redirect、REST、Session vs Token

HTTP 是無狀態協定，每個 Request 獨立、Server 不記狀態；靠 Session/Token 補上「登入狀態」。

方法語意

- GET查詢／POST新增／PUT整筆覆蓋／PATCH局部更新／DELETE刪除。GET/PUT/DELETE 幂等（重複呼叫結果相同），POST 不幂等。

狀態碼

- 2xx成功／3xx轉址(301永久,302暫時)／4xx客戶端錯(401未登入,403無權限,404找不到)／5xx伺服器錯。

Redirect 原理（考古題）

- Server 回 3xx 狀態碼＋Header 的 Location 帶新網址，瀏覽器自動對新網址發新 Request。例：登入後 302 導回首頁。

RESTful 原則

- URL 用名詞代表資源（/users/1）、方法表達動作、Server 端無狀態。

Session vs JWT

- Session存Server端、需查表、可即時撤銷；JWT整包簽章交給Client、免查表易擴展、較難即時作廢。

選擇/是非	簡答/開放	程式實作
配對狀態碼與情境（401沒登入 vs 403無權限）。	考古題「說明 HTTP redirect 如何運作」→ 答：3xx狀態碼＋Location header＋瀏覽器自動發新請求。	可能考寫 redirect API：FastAPI 用 RedirectResponse、Flask 用 redirect()。

二 作業系統與併發基礎

Process/Thread、Race Condition、Lock、Deadlock

CPU 用時間切片交錯執行製造「同時執行」的錯覺；共用記憶體才會有搶資源的問題。

Process

- 獨立記憶體空間，Process間互不干擾，溝通需IPC。

Thread

- 同Process內共用記憶體，溝通快但易搶資料。一句話：Process各自獨立房子，Thread是同房子共用冰箱的家人。

Race Condition（考古題延伸）

- 多執行緒同時讀寫共享資料，結果因執行順序而異。例：`count+=1`
其實是讀取→運算→寫回三步驟，中間被插隊就出錯。

Critical Section / Lock

- 會存取共享資源的區塊需上鎖，同時間只能一方進入。

Deadlock 四要件

- 互斥／佔有並等待／不可搶奪／循環等待，四者同時成立才會死鎖。

Redis 分散式鎖

- `SET key value NX`，key不存在才設值成功，多機情境下用來跨伺服器搶鎖，設TTL防止當機卡死。例：搶票座位鎖。

選擇/是非	簡答/開放	程式實作
是非題：Thread共用記憶體、Process不共用。	考古題「介紹 Process/Thread」→ 背上方定義＋比喻，加一句「我用Redis SET NX處理搶票高併發座位衝突」。	可能考模擬鎖或計數器安全性（多執行緒 <code>count+=1</code> 為何出錯）。

三 Git 版本控制

Commit/Branch/HEAD、Merge vs Rebase

Commit是快照，Branch是指向commit的指標（不複製檔案），HEAD指向你目前所在位置。

Commit

- 一次變更的快照，唯一hash值。

Branch（考古題核心）

- 指向某commit的可移動指標，建立成本極低。

HEAD（考古題核心）

- 指向目前所在branch或commit。

考古題敘述

- 「A branch is a reference to a commit, and HEAD is a reference to the current commit or branch you have checked out.」→ True。

Merge vs Rebase

- Merge保留兩邊歷史+新增合併commit；Rebase把commit接到目標分支後面、歷史乾淨但改寫hash，不對已推送的共用紀錄做rebase。

三態

- Working Directory → git add → Staging Area → git commit → Repository。

選擇/是非	簡答/開放	程式實作
是非/選擇最常見，重點是branch＝指標不是複製。	可能問如何處理merge conflict，可提MineMarket四人協作經驗。	較少考code，可能問「用什麼指令解決情境」。

四 資料庫基礎（SQL／NoSQL／Redis）

ACID、Index、正規化、Redis資料結構

SQL固定schema、支援複雜關聯查詢；NoSQL結構彈性、易水平擴展；Redis放記憶體所以快。

SQL vs NoSQL

- SQL資料結構固定、關聯查詢+交易一致性強；NoSQL結構彈性、易擴展。例：Threat Intel Agent因情資來源欄位不固定而選MongoDB。

ACID

- Atomicity原子性(全成功或全失敗)／Consistency一致性／Isolation隔離性／Durability持久性。

Index

- 類似書本目錄，B-Tree加速查詢，代價是拖慢寫入+佔空間。

正規化 Normalization

- 拆表消除重複，但過度正規化增加JOIN成本。

Redis資料結構

- String(快取/計數器)、Hash(物件屬性)、List(佇列)、Set(去重)、Sorted Set(排行榜)。

TTL與Cache-aside

- TTL設過期時間；Cache-aside=查快取沒有才查DB並寫回快取。

選擇/是非	簡答/開放	程式實作
ACID四要素配對、SQL/NoSQL特性判斷。	「為何MineMarket用MySQL、Threat Intel Agent用MongoDB」→用「資料結構固定性」回答。	可能考基本SQL(SELECT/JOIN/WHERE)或Redis指令。

五 後端框架與非同步機制

async/await、WSGI vs ASGI、FastAPI vs Flask

非同步靠Event Loop：遇到I/O先讓出控制權處理別的任務，I/O完成再回來繼續，不需為每個連線開Thread。

同步 vs 非同步

- 同步：排隊等待。非同步：I/O時讓出控制權做別的事，適合I/O bound場景。

WSGI vs ASGI

- WSGI(Flask預設)同步、一次一請求；ASGI(FastAPI+Uvicorn)非同步、少資源扛大量連線。

FastAPI vs Flask

- FastAPI：Pydantic驗證+自動文件+原生async，適合高併發API。Flask：輕量彈性，適合SSR樣板渲染。

Middleware/Decorator

- 統一處理身份驗證/Log/例外的邏輯層。例：MineMarket自製路由守衛、參數解析、交易三類Decorator。

ORM

- 把資料表對應成程式物件，免手刻SQL，底層仍轉譯成SQL執行。

選擇/是非	簡答/開放	程式實作
FastAPI vs Flask差異配對題。	「為何搶票系統選FastAPI、MineMarket選Flask」→ 併發需求 vs SSR需求。	可能考寫簡單endpoint或middleware/decorator。

六 系統設計基礎（高併發／擴展性）

Cache策略、限流、水平擴展、Message Queue

系統設計是資源有限、流量無限的取捨；Cache用空間換速度，限流犧牲部分請求換系統不被壓垮。

Cache三陷阱

- 快取穿透(查不存在的資料每次打DB)、快取擊穿(熱門key過期瞬間大量請求打DB)、快取雪崩(大量key同時過期壓垮DB)。

Rate Limiting

- Token Bucket：固定速率產生令牌，請求需消耗令牌，避免瞬間流量壓垮系統。

水平 vs 垂直擴展

- 垂直=升級單機規格(有上限)；水平=增加主機數(需無狀態設計+負載平衡)。

Message Queue

- 任務丟佇列非同步消化，達到解耦+削峰。例：MineMarket玩家離線補發道具走排隊補發。

實戰對應

- 搶票系統：Redis SET NX鎖+壓測10~600人找出race condition+根因分析→修補→回歸驗證。

選擇/是非	簡答/開放	程式實作
較少出現，可能考快取策略判斷。	「設計高流量系統」中論題 → 套用搶票系統Redis鎖+壓測+race condition修補流程。	較少考code，若有多為限流/去重簡化版。

七 AI／LLM／Agent 應用＋Prompt Engineering

職缺核心：Token/Tool Use/Agent決策/Prompt技巧

LLM是自回歸生成(一次預測下一個Token)；Tool Use讓模型能呼叫外部API取得即時資訊再繼續推理。

Token與Context Window

- Token是處理單位；Context Window是模型一次能看到的Token上限。

Tool Use/Function Calling

- 模型判斷需要外部資訊時輸出結構化工具呼叫請求，程式執行後把結果餵回模型。

Agent vs Rule-based

- Rule-based寫死規則；Agent由LLM自主判斷查什麼、查幾次、何時足夠。

強制結構化輸出

- 用tool_choice強制第二輪呼叫回報函式，保證輸出可解析JSON。例：Threat Intel Agent兩輪Tool Use架構。

速率限制與快取

- 外部API有呼叫次數限制(如VirusTotal每分鐘4次)，搭配TTL快取降低重複呼叫。

決策可追溯性

- 記錄Agent每步查了什麼/耗時/是否命中快取，供事後稽核。

Prompt技巧：Zero/Few-shot

- Zero-shot不給範例；Few-shot給幾組輸入輸出範例讓模型模仿格式。

Prompt技巧：CoT思維鏈

- 要求模型一步步推理再給答案，提升複雜推理準確度，但耗更多Token。

Prompt技巧：角色設定/格式限制

- 給身分情境引導語氣；明確指定輸出JSON schema減少解析失敗。

Prompt技巧：任務拆解/負面指引/迭代

- 拆解成多步驟指令；明講「不要做什麼」防止幻覺；反覆調整措辭收斂輸出。

Temperature

- 數值低輸出穩定保守(適合結構化抽取)，數值高輸出多樣但不穩定(適合發想)。

選擇/是非	簡答/開放	程式實作
Prompt/Token/Tool Use名詞定義配對。	申論題重點戰場，直接用Threat Intel Agent案例：兩輪Tool Use+強制結構化輸出+決策軌跡+風險判斷執行分離。	較少考code，可能考簡化版tool-calling流程或prompt撰寫。

八 程式邏輯與演算法基礎

對應Coding Challenge（Math Challenge），純手寫

2題皆Math Challenge，系統偵測切分頁/複製貼上，需當場手寫。下筆前先估資料量級再選解法。

Big O

- $O(1)$ 常數／ $O(\log n)$ 對數／ $O(n)$ 線性／ $O(n \log n)$ 排序／ $O(n^2)$ 巢狀迴圈／ $O(2^n)$ 未優化遞迴。

質數判斷

- 檢查到 $\sqrt{n}$ 即可，不必到 n 。

階乘/費氏數列

- 階乘迴圈或遞迴。費氏：原始遞迴 $O(2^n)$ 重複計算子問題；記憶化遞迴 $O(n)$ 查表；迭代滾動變數 $O(n)$ 時間 $O(1)$ 空間，效率最佳。

陣列/字串

- 排序用內建sort；去重用Set；統計次數/找重複字元用Dictionary/HashMap， $O(n)$ 遠勝雙迴圈 $O(n^2)$ 。例：banana逐字掃描累加次數，a:3,n:2為重複字元。

GCD/LCM

- $GCD(a,b)$: $b=0$ 回傳 a ，否則遞迴 $GCD(b, a \bmod b)$ 。 $LCM=a \times b \div GCD(a,b)$ 。

下筆前檢查

- 先想邊界條件($n=0$ /負數/空輸入)，再想複雜度，最後才寫code。

選擇/是非	簡答/開放	程式實作
不適用（此類為程式實作題）。	不適用（此類為程式實作題）。	考前手寫練2-3題（質數/費氏/GCD/找重複字元），不能依賴查詢，因系統偵測切分頁。

演算法	原理	複雜度	這次測驗可能性
質數判斷	試除法到 $\sqrt{n}$	$O(\sqrt{n})$	高
GCD/LCM	輾轉相除法	$O(\log n)$	高
費氏/階乘	遞迴/記憶化/迭代	$O(n) \sim O(2^n)$	高
字串找重複字元	HashMap計數	$O(n)$	高
迴文/字串反轉	雙指標比對	$O(n)$	中高
雙指標 Two Pointers	指標相向/同向移動	$O(n)$	中
二分搜尋	每次排除一半範圍	$O(\log n)$	中(需已排序)
排序原理	理解即可、用內建sort	$O(n \log n)$	低(手刻)/中(觀念)
0/1背包 DP	$dp[i][w] = \max(\text{不拿}, \text{拿})$	$O(n \times \text{容量})$	中低
矩陣鏈乘法	DP枚舉切割點	$O(n^3)$	低
旅行商TSP	NP-Hard, Held-Karp/近似解	$O(n^2 \times 2^n)$	低(多為概念題)

矩陣鏈乘法/TSP 屬課程經典DP/NP-Hard題型，時限內完整實作機率低，但建議能口述原理：矩陣鏈乘法枚舉切割點求最少乘法次數；TSP用bitmask狀態壓縮DP或退而求其次用近似解。

九 後端安全基礎

SQL Injection、XSS、密碼雜湊、JWT

根本問題是「使用者輸入被當成指令執行」；防範原則是把資料與語法/邏輯分開處理。

SQL Injection與防範

- 字串拼接SQL會被惡意輸入攻破；用參數化查詢/Prepared Statement讓輸入值永遠只是資料。

XSS

- 惡意script注入頁面，其他使用者瀏覽時被執行；輸出內容需escape逸出處理。

密碼儲存

- 不可明文存，需bcrypt等不可逆雜湊+加鹽(salt)，防rainbow table查表攻擊。

JWT重點

- 簽章需驗證避免竄改、有效期不宜過長、payload只編碼未加密不可放敏感資料。

選擇/是非	簡答/開放	程式實作
SQL Injection/XSS基本定義判斷。	「怎麼確保系統安全性」→可提MineMarket ECPay CheckMacValue簽章驗證經驗。	較少直接考code。

十 機器學習與深度學習基礎

傳統ML／DL神經網路／LLM-NLP，AI工程師知識底盤

ML本質是用資料找出函數 $f(x)=y$ ；訓練靠Loss Function衡量誤差、梯度下降調整參數收斂。

三大學習典範

- 監督式(有標準答案)／非監督式(找內在結構)／強化學習(獎懲訊號學策略)。

Overfitting/Underfitting

- 過擬合=背下訓練資料細節、新資料表現差(緩解：正則化/增加資料/Early Stopping)；欠擬合=模型太簡單學不好。

評估指標

- Precision=TP/(TP+FP)重視不誤判；Recall=TP/(TP+FN)重視不漏抓；F1=兩者調和平均。

傳統演算法

- Linear/Logistic Regression、Decision Tree、Random Forest(集成學習)、KNN、K-means(分群)、SVM(最大間隔超平面)。

神經網路核心

- 神經元加權求和+激活函數(ReLU/Sigmoid/Softmax)；反向傳播用鏈鎖法則算梯度，梯度下降更新參數。

CNN/RNN-LSTM

- CNN用卷積核提取局部特徵、參數共享，擅長影像；RNN/LSTM處理序列資料、具記憶能力，現多被Transformer取代。

Tokenization/Embedding

- 文字切成Token；Embedding把Token轉高維向量，語意相近距離相近。

Transformer/Self-Attention

- 處理某Token時同時權衡句中所有Token關聯度，捕捉長距離語意關係。

Pretraining/Fine-tuning/RAG

- 預訓練學通用語言規律；微調用少量任務資料調整模型；RAG回答前先查外部知識庫再生成，補足知識截止限制。

Hallucination幻覺

- 模型缺乏依據仍自信生成錯誤內容。例：Threat Intel Agent用強制結構化輸出+決策記錄提高可信度。

選擇/是非	簡答/開放	程式實作
Overfitting/Precision/Recall/Token/Embedding名詞配對。	高機率——104訊息會評估AI工具運用能力，建議用Threat Intel Agent案例回答Transformer/RAG/Hallucination概念題。	低機率——Math Challenge型測驗不太會考手刻ML演算法。

十一 Docker 容器化

Image/Container/Dockerfile/Volume

把應用程式+執行環境打包成映像檔，任何機器上執行結果一致；靠Namespace+Cgroups做輕量隔離，比VM省資源。

Image vs Container

- Image是唯讀模板，Container是運行中的實體，一個Image可開多個Container。

Dockerfile

- FROM基底映像、COPY複製檔案、RUN安裝指令、CMD啟動指令。

Volume

- 掛載外部持久化儲存，讓資料跨容器重啟/刪除保留。例：對應Heroku檔案系統無狀態問題的另一解法。

Docker Compose

- yaml定義多容器（API+DB+Redis）一起啟動。

你的實戰

- Threat Intel Agent以Docker容器化部署到雲端平台。

選擇/是非	簡答/開放	程式實作
Image vs Container、Docker vs VM差異判斷。	「為什麼你的專案用Docker部署」→環境一致性、避免重新架設時漏裝套件版本。	較少考Dockerfile撰寫，但可能被追問細節。

十二 CI/CD 與 DevOps 基礎

CI持續整合、CD持續交付/部署、Pipeline

CI在合併前自動測試盡早抓錯；CD把通過測試的程式碼自動(或半自動)部署，縮短上線風險與時間。

CI

- 推送後自動Build+Test，常見工具GitHub Actions/GitLab CI/Jenkins。

CD: Delivery vs Deployment

- Delivery=打包成可部署版本但仍需人工確認上線；Deployment=全自動直接上線。

Pipeline常見步驟

- Lint→Build→Test→Deploy，依序執行、失敗則中止。

環境分層

- Development→Staging(模擬正式環境)→Production，逐層驗證降低風險。

你可延伸

- Railway/Heroku/Render部署經驗本質屬CD範疇；可提「正在補強Docker/CI-CD」呼應自傳方向。

選擇/是非	簡答/開放	程式實作
CI vs CD定義配對、常見工具名稱。	「你了解CI/CD嗎」→可提MineMarket情境：PR時自動跑測試、沒過不能合併，避免多人協作互相影響。	幾乎不會考實作，屬概念理解題。

附錄 專有名詞速查表

依章節分類，找不到定義時查這裡

一、HTTP／Web 通訊

HTTP 超文本傳輸協定，瀏覽器與伺服器之間傳輸資料的通訊協定。
TCP 可靠、有連線的底層網路傳輸協定，HTTP 建立在其上。
Header HTTP 訊息中夾帶的中繼資訊，如格式、認證資訊。
Status Code 伺服器回應的三位數狀態碼，如 200、404。
Redirect（轉址） 伺服器要求瀏覽器改連到另一個網址，用 3xx 狀態碼表示。
Idempotent（冪等） 同一請求重複呼叫多次，結果都相同。
RESTful 以資源為 URL 主體、用 HTTP 方法表達動作的 API 設計風格。
Session 登入狀態存於伺服器端，需查表確認身份。
Token／JWT 登入狀態加密簽章後交由客戶端保存的憑證。

二、作業系統與併發

Process（行程） 作業系統配置資源的基本單位，各自獨立記憶體空間。
Thread（執行緒） Process 底下的執行單位，同 Process 內共用記憶體。
Concurrency（並發） 單核心交錯執行多任務，造成同時執行的錯覺。
Parallelism（平行） 多核心真正同時執行多個任務。
Race Condition 多執行緒同時讀寫共享資料，結果因執行順序而異。
Critical Section（臨界區） 會存取共享資源、需避免同時被多方進入的程式區塊。
Lock／Mutex 確保同一時間只有一方能進入臨界區的機制。
Deadlock（死鎖） 多個程序互相等待對方釋放資源，永遠卡住。
IPC 行程間通訊（Inter-Process Communication）。
TTL 資料的存活時間，超過即失效。

三、Git 版本控制

Commit 一次程式碼變更的快照，有唯一 hash 值。
Branch 指向某個 commit 的可移動指標，非複製檔案。
HEAD 指向目前所在的 branch 或 commit。
Merge 合併兩條分支歷史，新增一個合併 commit。
Rebase 把 commit 依序接到目標分支後面，改寫 commit hash。
Staging Area（暫存區） git add 之後、commit 之前的中介區。
Conflict（衝突） 同一段程式碼被兩邊修改，需人工判斷取捨。

四、資料庫

SQL／NoSQL 關聯式資料庫（結構固定）／非關聯式資料庫（結構彈性）。
ACID 資料庫交易的四項保證：原子性、一致性、隔離性、持久性。
Index（索引） 加速查詢的額外資料結構，多為 B-Tree。
Normalization（正規化） 拆表消除資料重複，代價是增加 JOIN 成本。
JOIN 合併多張資料表查詢結果的 SQL 操作。
Cache-aside 先查快取、沒有才查資料庫並寫回快取的策略。
Redis 記憶體級資料庫，常用於快取、鎖、佇列。

五、框架與非同步

Sync／Async 同步（依序等待）／非同步（I/O 時讓出控制權處理別的任務）。
Event Loop 非同步程式監控多個待完成任務、輪流處理的機制。
I/O bound／CPU bound 任務瓶頸在等待輸入輸出／在運算本身。
WSGI／ASGI Python 同步／非同步的 Web 伺服器標準介面。
Middleware 請求進入實際邏輯前後統一處理的邏輯層。
Decorator 不改動原函式、外層包裝額外邏輯的 Python 語法。
ORM 把資料表對應成程式物件的技術。

六、系統設計

快取穿透／擊穿／雪崩 快取失效機制設計不當，導致資料庫被大量請求打爆的三種情境。
Rate Limiting（限流） 限制單位時間內的請求量。
Token Bucket（令牌桶） 固定速率發放令牌、控制請求速率的演算法。
水平擴展／垂直擴展 增加主機數量／升級單機規格。
Message Queue（訊息佇列） 任務先進佇列、由後端非同步消化，達到解耦與削峰。

七、AI／LLM／Agent

Token LLM 處理文字的最小單位。
Context Window 模型一次能讀取的 Token 上限。
Zero-shot／Few-shot 不給範例／給少量範例讓模型模仿格式。
Chain-of-Thought (CoT, 思維鏈) 要求模型先逐步推理, 再給出最終答案。
Tool Use／Function Calling 讓 LLM 呼叫外部工具/API 取得即時資訊或執行動作。
Agent 由 LLM 自主判斷決策流程的系統, 而非事先寫死規則。
Hallucination (幻覺) 模型缺乏依據時, 仍自信生成看似合理但錯誤的內容。
Temperature 控制模型生成隨機性的參數, 越低越穩定、越高越多樣。

八、演算法

Big O 衡量演算法時間/空間隨資料量成長速度的記號。
Brute Force (暴力解) 窮舉所有可能的解法。
Recursion (遞迴) 函式呼叫自己處理更小的子問題。
Memoization (記憶化) 快取已算過的結果, 避免重複計算。
Two Pointers (雙指標) 用兩個指標依條件相向或同向移動掃描資料。
Dynamic Programming (DP) 把大問題拆成重疊子問題並記錄結果求解。
NP-Hard 目前沒有已知多項式時間演算法能保證找到最佳解的問題類型。

九、後端安全

SQL Injection 惡意 SQL 語法混入輸入欄位, 造成資料庫被操控。
XSS 惡意 script 被注入頁面, 其他使用者瀏覽時被執行。
Prepared Statement (參數化查詢) 語法結構與參數值分開處理, 防止注入。
Salt (加鹽) 雜湊密碼前混入隨機字串, 防止查表攻擊。

十、機器學習／深度學習

Overfitting／Underfitting 過擬合 (背下訓練資料、新資料表現差)／欠擬合 (模型太簡單學不好)。
Precision／Recall／F1 精確率 (別誤判)／召回率 (別漏抓)／兩者的調和平均。
Backpropagation (反向傳播) 從輸出端往回逐層計算每個參數梯度的演算法。
Gradient Descent (梯度下降) 沿損失函數梯度反方向微調參數, 逐步逼近最小損失。
Embedding (詞嵌入) 把文字轉成高維度向量, 語意相近則向量距離相近。
Transformer／Self-Attention 現代 LLM 核心架構, 讓模型權衡句中所有詞彼此的關聯。
Fine-tuning (微調) 用少量任務資料進一步調整已訓練好的模型。
RAG 回答前先檢索外部資料再生成, 補足模型知識限制。

十一、Docker

Image (映像檔) 唯讀的應用程式打包模板, 本身不執行。
Container (容器) Image 被啟動後、實際運行中的實體。
Dockerfile 描述如何一步步建構出 Image 的設定檔。
Volume (資料卷) 掛載外部持久化儲存, 讓資料跨容器重啟/刪除保留。

十二、CI/CD／DevOps

CI (持續整合) 程式碼推送後自動觸發建構與測試。
CD (持續交付／部署) 通過測試的程式碼自動 (或半自動) 部署上線。
Pipeline (流水線) 定義一連串自動化步驟 (Lint/Build/Test/Deploy) 的設定。
Staging (預備環境) 盡量模擬正式環境的測試環境, 上線前的最後一關。